CE/CS/SE 3354 · Software Engineering · Fall 2026
Project Milestone P2

Analysis and High-Level Design

Whoosh Events

Team	Team 05, section .003
Document version	1.0 (baseline for P2; P1 was version 0.1)
Date	November 8, 2026
Repository URL	https://github.com/AO2-Git/CS3354-Team05-Project
Git tag for this milestone	p2
Submitted on Canvas by	Austin Ou Yang

Team member	NetID	GitHub username
Austin Ou Yang	axo240008	AO2-Git
Habib Mashhood	hxm240043	hmashhood
Jaiom Rola	jmr230006	jaiom-rola-sp
Ahmed Fouad	amm230041	ahmedfouadnew
Atharva Thombare	axt230126	Ath-T
Sinan Can Demir	dal715992	sinan-can-demir

0 Revision History, Contribution Record and Changes Since P1

All three tables below are graded. The contribution record carries the individual 20% of this milestone and must agree with your repository history.

Version	Date	Author	Summary of change

0.1	10/18/2026	Team 05	P1 baseline: proposal and requirements
1.0	11/08/2026	Team 05	P2 baseline: analysis and high-level design added

0.1 Individual contribution record (P2)

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
Austin Ou Yang	Section 6.3, ADR-03, 6.3 Box Diagram, GitHub docs folder, Section 6.4, 6.4 Diagram	PR#4, Commits 4a17667, e8ad637, 8cc58ab, e6e95ca	
Habib Mashhood	6.1 quality-attribute scenarios (QAS-01 to QAS-03), 6.2 architectural style and alternatives, ADR-01 (architectural style) and the ADR-01 row of 7.2	PR #10 (branch p2/habib-architecture); commit ac0a6c2	PR #4 (reviewed and merged)
Jaiom Rola	Spike S-01 (scraper code, README and the numbers used in section 8), ADR-02 (event refresh strategy), 10.3 risks, 11 open questions; React starter for P3	PR #5 (branch spike/s01-event-scraper); commits 0724054, d99de97, 6197c3e, d2180e1 and 2755b54 (spike), ca944a2 (React starter), 1dff371 (hourly refresh in README)	
Ahmed Fouad	0.2 changes CH-01 to CH-04; P1 fixes in 2.2 (student poll of 7), 3.1 to 3.5 (FR-02, FR-04, QR-02, use case diagram, actor-goal list); 4.1 to 4.3 (UC-01, UC-02, UC-03, self-review log); 5.1 domain model; 5.2.1 and	Issue #1; PR #2 (branch p2/ahmed-analysis-models); commits ee07f0d (diagram sources, docs/p2/diagrams), 27d6550 (section text, docs/p2/sections), 83927f7 (P1 fixes in 1.1 to 2.6, context diagram), 31fa6c3 (sequence diagram notation) and 2ac94e6 (UC-01	PR #9, PR #6

	5.2.2 scenario tables and sequence diagrams	postcondition). PR #2 merged into preproduction as 21fd87d	
Atharva Thombare	Section 5.3: Cross checked analysis sequence diagrams with the UC tables in 4.2. Section 9.1, verified the requirements for traceability with their UC, components and QAS/ADR. 9.2, Gap analysis table. Reviewed and commented feedback on PR #2	Commits: 91541ac, 95e1abe, 7c293ed, 9605a70, a8c04c9, efea277 PR #9 PR #12	PR #2
Sinan Can Demir	Sections 6.5 (analysis objects to components), 6.6 (design principles applied), 7.2 (trade-off summary), 10.2 (plan to P3) and 10.5 (AI-use declaration, collected from the team). Collected the signatures for section 12. Reviewed teammates' work: S-01 spike (PR #5) and the analysis models (PR #2).	PR #6, branch p2/sinan-summary-tables, commit d0935fd (draft of the sections above), 01ca14e and cd4ccbd. Review comments on PR #5 and a comment on PR #2.	PR #5 PR #2

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01	UC-01 rewritten turn by turn in 4.2 with step (0), alternate flows and an acknowledgment	P1 feedback C3: main flow was a single line, alternate flow one sentence	5.2.1 sequence diagram is built from it	Team meeting , Nov 4

CH-02	Second and third expanded use cases added: UC-02, UC-03 (4.2)	P1 feedback C3: only one use case was written out in full	5.2.2 sequence diagram is built from UC-02; 9.1 traces FR-10 to it	Team meeting , Nov 4
CH-03	Section 3.5 matrix replaced by a pointer to 9.1; every requirement now maps to a use case ID (FR-01 to FR-12, QR-01 to QR-06, CON-01 to CON-04)	P1 feedback C3: matrix pointed to phrases, and UC-01 claimed all twelve FRs	9.1 extends the chain to classes, components and ADRs	Team meeting , Nov 4
CH-04	Acceptance tests of FR-02 and FR-04 rewritten with a number and a pass condition; QR-02 reworded with a measurable threshold in the same form as QR-01; IDs unchanged	P1 feedback C2: not every requirement had a concrete acceptance test	QAS-01 in 6.1 cites the revised QR	Team meeting , Nov 4
CH-05	Section 2.2 adds answers from UTD students on how they find events today; 2.6 adds the S-01 scrape result	P1 feedback C1: need and feasibility were assumptions	Feasibility verdict now rests on measured evidence	Team meeting , Nov 4
CH-06	Repository given a structure (/docs/p2, /docs/p2/diagrams, /tests/spikes); each member committed their own sections on their own branch and opened a pull request into preproduction	P1 feedback C4/C5: repository held only the README, so contributions could not be verified	0.1 cites commit hashes and PR numbers	Team meeting , Nov 4
CH-07	Roles in 10.1 stated as responsibilities instead of section numbers	P1 feedback C4	None	Team meeting , Nov 4
CH-08	Every milestone in 10.2 has one named owner	P1 feedback C4: five of seven milestones were owned by "Team 5 Members"	None	Team meeting , Nov 4
CH-09	Risks in 10.3 rewritten for P2 (source layout change, incomplete event data, merge	P1 feedback C4: R5 was blank	The layout-change risk is covered by the S-01	Team meeting , Nov 4

	conflicts), each with an owner and a mitigation		fallback to last good data (ADR-02)	
CH-10	AI-use declaration completed by every member (10.5)	P1 feedback C4: declaration was empty	Commits with an AI co-author line are matched in 10.5	Team meeting , Nov 4
CH-11	All template guidance, example and warning boxes removed before export	P1 feedback C4: guidance boxes were left in	None	Team meeting , Nov 4
CH-12	Open questions in 11 replaced with Q1 to Q3 from the spike, each with an owner and a due date	Raised by spike S-01 and ADR-02	The assumptions in 11 hold until the Nov 16 due dates	Team meeting , Nov 4

1 to 3 Introduction, Overall Description and Requirements (carried forward)

1 Introduction

1.1 Purpose of this document and of the system

Whoosh Events is a website that gathers UTD's events (clubs, social, community) into one list, and shows students where to submit their own events on the official source sites. Currently, information about UTD Events is scattered across multiple websites, so our goal is to consolidate that information into one website.

1.2 Scope

In scope — we will build this	Out of scope — we will not, and why

Scraper that finds all events from multiple sources	No comments over event details
Loaded list[block style] of events with filter	Direct event creation
RSVP Option/ add to personal calendar	Calendar view of events
A page that teaches you how to create a UTD event (club, community, social)	Log in page
Event card- card template that fits the information relevant to the specific event	Event Recommendation

Why the exclusions were excluded:
Direct event creation - To avoid events that were planned without permission, we decided to avoid having direct event registration
No comments over event details - This should fall on the Event Organizer to make their contact info very clear for any questions attendees may have.
Calendar view of events - Calendar view can make browsing difficult; list view will make it easier for users to find events they are interested in.

No log in page - not required to see public events and no security is needed to look at the number of events as a whole.
No event recommendation - Users should be able to know what they like or dislike. The goal of our website is to consolidate information for easier access, not to make recommendations.

1.3 Definitions, acronyms and abbreviations

Term or abbreviation	Meaning as used in this project
Event	A scheduled UTD activity (club, social , or community) listed on a source site in 1.4
Event Card	A fixed layout block that shows one event details in the event list
Event List	The scrollable list of event cards on the main page
Tag	A category or label on an event (for example “social” or “career”) that users can filter by. Our Label list is Social, Academic, Career, Arts & Culture, Sports & Recreation, Service & Volunteering, Cultural.

Scraper	The component that automatically reads event pages from the source sites and extracts event data.
Event organizer	the club, department, or person who posts an event on a source site.
Source site	An external site the scraper reads: the UTD calendar, or the Nebula labs club Calendar.
RSVP	A user's signal that they plan to attend an event

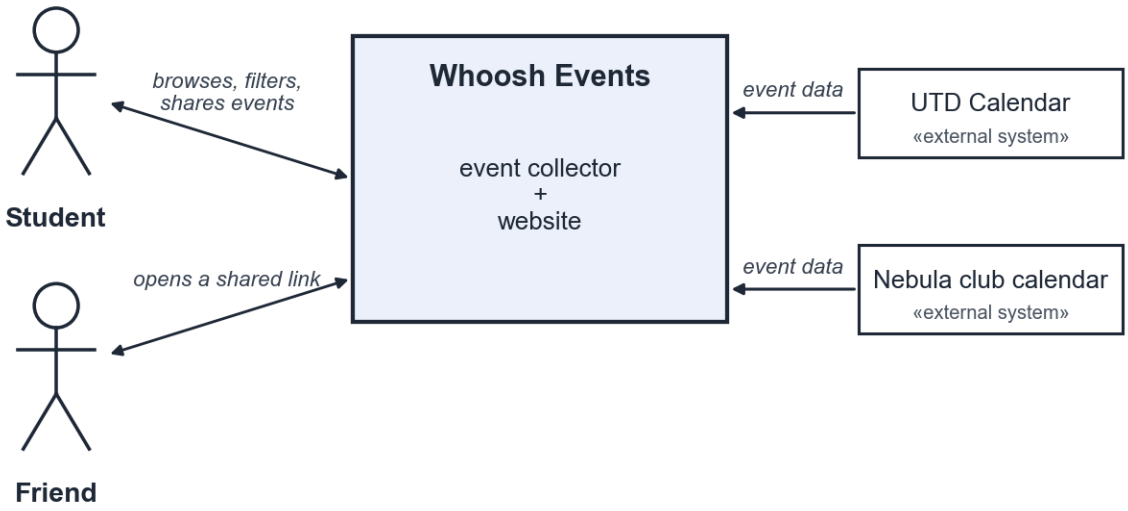
1.4 References

[R1] UTD Calendar - https://calendar.utdallas.edu/calendar
[R2] Nebula Labs Club Calendar - https://clubs.utdnebula.com/events?page=1

2 Overall Description

2.1 Product perspective and context

Figure 2.1 Context diagram



The boundary surrounds Whoosh Events itself: the event collector and the website, which are the only parts we build and maintain. Students and their friends use the website from outside the boundary. The UTD Calendar and the Nebula club calendar are external systems we only read from, under their robots.txt rules (CON-03); we did not build them and cannot change them.

2.2 Stakeholders and how you reached them

Stakeholder	Role in relation to the system	What they need from it	How you learned this

Site Users	Start a use case	One place that shows events from all UTD groups, including ones they do not already follow, early enough to attend, and filters to find what interests them	Poll of 7 UTD students on Nov 5, 2026 (How do you find out about UTD events?). Each relies on one channel: Instagram, a club Discord, a WhatsApp free-food group, walking past the Student Union, MSA announcements, the UTD Calendar or university emails; the official channels carry only academic and university events, and none mentioned Nebula. One hears of events only after they are over. So students see only the groups they already follow, and too late.
Developer Team	Builds and Maintains the Site	A scraper to grab information from UTD pages, and a website to display the information obtained in a formal layout.	This was decided on during the 10/14/26 Team 05 meeting. These are fundamental parts of our idea.
Event Organizers(clubs, departments)	Indirect user, content provider	Their events shown accurately, and a clear way to learn how to host one	From the source sites in 1.4: every event names its organizer, and each site has its own process for posting an event. FR-04 points organizers to that process instead of replacing it.

Nebula Labs and UTD Calendar Maintenance teams	Operate the source sites, never use our system	Scraping that doesn't overload or break their sites	From the sites' robots.txt files, read in Nov 2026: the UTD Calendar asks automated readers to wait 1 second between requests, and Nebula allows all pages. CON-03 follows these rules.
Course instructor & Teaching Assistants	Evaluate the project	Requirements that are testable and within scope	stated in the p1 template

2.3 User characteristics

Users are assumed to have basic knowledge to navigate the internet. They should already know that the site is for navigating a list of UTD events. They cannot be assumed to know what events are available. It may also be assumed that they are accessing the site from a laptop, desktop, or mobile device. Layouts must work on a phone screen, and the site should be readable for users with accessibility needs, for example keyboard navigation and sufficient text contrast.

2.4 Constraints

Event data comes only from the sources, which are the source sites (Nebula labs UTD clubs & UTD Events calendar). Users cannot create events directly on the site. P2 must be submitted by November 8, 2026. Work is managed in the team Github repo, with testing in preproduction before merging to production.

2.5 Assumptions and dependencies

#	Assumption or dependency	What breaks if it does not hold
A1	Every event on the source sites in 1.4 has a readable title, date and start time.	An event with no date cannot be placed in the list; fields that are only missing show Not listed (FR-07).
A2	Both source sites stay publicly accessible without login and don't block automated access.	The scraper returns no events, and the event list is empty or stale.
A3	The same event is not posted on both sources, or can be matched when it is.	The same event appears twice in the event list, which breaks FR-03.

2.6 Feasibility summary

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Feasible, provisional	Both source sites are public event pages (Nebula labs UTD clubs & UTD Events Calendar). The UTD Calendar also offers a public events feed, and both robots.txt files allow reading (2.2). Spike S-01 (section 8) measures a real scrape of both sites.
Operational	Feasible	The site is read-only and has no accounts, so there is little to operate once deployed.
Schedule	Feasible with risk	6 members, and the plan in 10.2 runs to the P3 deadline on December 9.
Economic / resource	Feasible	No budget is needed. It depends on the website being hosted for free and student time.

3 Specific Requirements

3.1 Functional requirements

ID	Requirement — one “shall” sentence	Priority	Source	Acceptance criterion
FR-01	The system shall have at least 20 events listed on one page.	MUST	1.2	A tester can confirm this by loading our website and counting the number of events listed on the page.
FR-02	The system shall have filters to display events by a specific “tag” from the list defined at 1.3	MUST	1.2 & 1.3	A tester picks 5 different tags from the list in 1.3 and filters by each one. Pass if, for all 5 tags, every event shown has the chosen tag and no upcoming event with that tag on the source sites in 1.4 is missing.
FR-03	An event that appears on more than one source site from our 1.4 list shall appear only once in the event list.	SHOULD	1.4, 2.2	A tester finds 3 events listed on at least two of sources listed on at 1.4. Pass if each appears exactly once in the event list.

FR-04	The system shall have a 'How can I host an event at UTD?' page that links to the official page for submitting an event on each source site in 1.4.	COULD	1.2	A tester opens the page and follows every link on it. Pass if the page has one link for each of the 2 source sites in 1.4 and each link opens that site's official page for submitting an event.
FR-05	The system shall show every event from source 1.4 in our event list.	MUST	1.2	A tester picks 10 upcoming events from each source of two sources from our sources list. Pass if all selected events appear in our event list.
FR-06	The system shall have an “Add to Calendar” feature that allows users to download .ics files.	COULD	1.2	A tester can confirm this by accessing any event on our website and clicking on the “Add to Calendar” icon. Then, confirm that the .ics file is downloaded.
FR-07	Each event card shall show the event's title, date, start time, location, organizer, tags and a link to the source page.	SHOULD	1.2	A tester checks 10 event cards, at least 3 from each source site. Pass if every card shows all seven fields, or 'Not listed' when the source site leaves a field blank.
FR-08	The system shall let a Site User filter the event list by event organizer.	SHOULD	2.2	A tester filters by 3 different organizers. Pass if every event shown each time belongs to that organizer.
FR-09	The system shall show an event’s full description when a Site User selects its event card.	SHOULD	2.2	A tester selects 5 event cards. Pass if each shows the same description as on its source site.

FR-10	Each event card shall have a “Copy link” button that copies a URL opening that event on our site.	COULD	2.2	A tester copies the link from 5 events and opens each in a new browser window. Pass if each opens the same event.
FR-11	The system shall load the next 20 events when a Site User reaches the end of the event list.	SHOULD	2.2	With more than 40 upcoming events, a tester scrolls to the end of the list. Pass if 20 more events appear.
FR-12	Each event card shall show the name of the source site the event came from.	COULD	2.2	A tester checks 10 event cards, at least 3 from each source site. Pass if each shows the correct source site name.

3.2 Quality requirements

ID	Requirement — with threshold, condition and method	Priority	Source	How it will be verified
QR-01	The event list page shall finish loading within 3 seconds on at least 19 of 20 page loads over a connection of 10 Mbps download with 50 ms latency.	SHOULD	2.2	A tester creates a custom Chrome DevTools network profile (10 Mbps download, 50 ms latency), loads the event list page 20 times and records each load time. Pass if at least 19 loads finish in 3 seconds or less.
QR-02	After a student chooses a tag, the filtered event list shall appear within 2 seconds on at least 19 of 20 tries over a connection of 10 Mbps download with 50 ms latency.	SHOULD	1.2	A tester uses the same Chrome DevTools network profile as QR-01, chooses a tag 20 times (rotating through 5 tags) and records the time from choosing the tag to the list appearing. Pass if at least 19 of the 20 finish in 2 seconds or less.

QR-03	An event posted on a source site shall appear in the event list within 2 hours for at least 9 of 10 newly posted events.	MUST	2.2, 5	A tester records when 10 new events are posted on the source sites in 1.4 during one week, then checks when each first appears in our event list. Pass if at least 9 appear within 2 hours.
QR-04	At least 19 of 20 randomly sampled events shall show the same title, date, time and location as on their source site.	MUST	2.2	A tester picks 20 events at random from our event list and compares the four fields with the source site. Pass if at least 19 matches on all four.
QR-05	Every page shall display without horizontal scrolling on a screen 375 pixels wide.	SHOULD	2.3	A tester opens every page in Chrome DevTools device mode at 375 pixels wide. Pass if no page needs horizontal scrolling.
QR-06	When a source site cannot be reached during a refresh, the event list shall keep all of that site's events from the last successful refresh.	SHOULD	2.5	A tester blocks one source site's address and runs a refresh. Pass if every event from that site that was listed before the refresh is still listed after it.

3.3 Constraint requirements

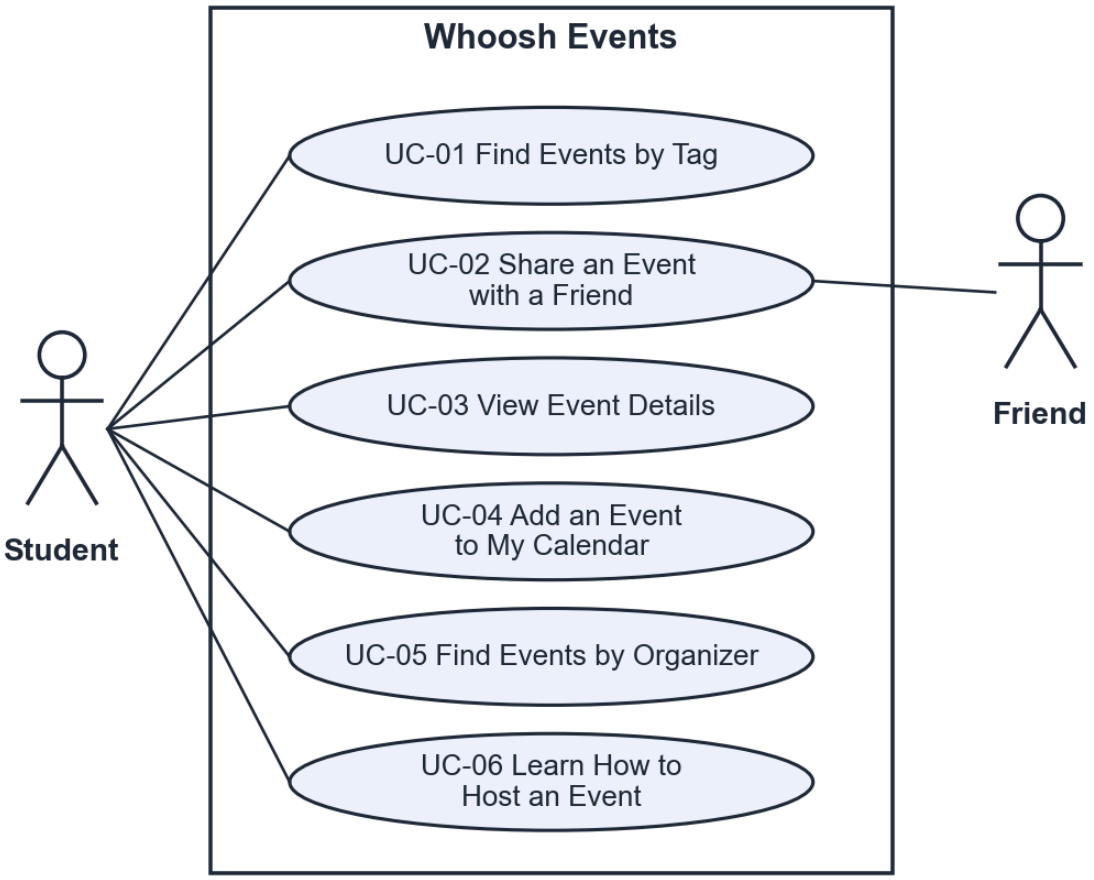
ID	Constraint — one sentence	Source	Why it cannot be negotiated
CON-01	The system shall not allow Site Users to create their own events.	2.4, 1.2	UTD events must be approved through UTD's own process before they are posted. Our site has no way to check that approval, so allowing event creation would let unapproved events appear as if they were official.

CON-02	The system shall get event data only from the source sites listed in 1.4.	2.4	The UTD Calendar and the Nebula Labs Club Calendar are UTD's official places for Uposting events. Taking events from any other site could show unapproved or fake events.
CON-03	The scraper shall read only the pages that each source site's robots.txt file allows.	2.2, 2.5	The owners of the source sites set these rules, not us. Ignoring them could get our scraper blocked, which would break assumption A2 and leave the event list empty.
CON-04	The system shall run on a free hosting service with no paid costs.	2.6	The team has no budget for this project, as stated in the economic feasibility verdict in 2.6.

3.4 Initial use cases

Figure 3.4 Use case diagram (actor-goal level)

m



Redrawn since P1 with the use case IDs used in sections 4, 5 and 9. The P1 diagram showed the scraper and the event organizers; the scraper is background work no actor sees (see 4.3), and organizers post on the source sites, not on our system. The three use cases marked “expanded” are written out turn by turn in 4.2.

UC-01 Find Events by Tag. Student finds upcoming events on a topic. FR-01, FR-02, FR-05, FR-07, FR-11. Expanded.

UC-02 Share an Event with a Friend. Student sends one event to a friend, who opens it. FR-07, FR-09, FR-10, FR-12. Expanded.

UC-03 View Event Details. Student sees everything about one event. FR-03, FR-07, FR-09, FR-12. Expanded.

UC-04 Add an Event to My Calendar. Student keeps an event in their own calendar. FR-06.

UC-05 Find Events by Organizer. Student sees what one club or office is running. FR-08.

UC-06 Learn How to Host an Event. Student finds where to submit their own event on each source site. FR-04.

3.5 Requirements to use case traceability

The P1 matrix is replaced by the full traceability table in section 9.1, which maps every requirement to a use case ID and on to classes, components and decisions.

4 Use-Case Analysis

Chapter 7, first half. Here you take the use cases from section 3.4 and write the most important of them out turn by turn, still with the technology switched off.

4.1 Use cases selected for analysis

Use case	Name	Why it was selected	Non-trivial steps
UC-01	Find Events by Tag	Carries most of the product: the main reason students visit	Steps 0, 2, 4
UC-03	View Event Details	Touches the most event fields (FR-03, FR-07, FR-09, FR-12, QR-04), including events merged from both source sites	Step 2
UC-02	Share an Event with a Friend	Two people and a link that can go out of date: the event can end, be removed or change before the friend opens it; tests QR-03 and QR-04	Steps 2, 4

4.2 Expanded use cases

UC-01 Find Events by Tag

Primary actor	Student
Supporting actors and external systems	None during this use case (events were collected from UTD Calendar and Nebula beforehand)

TUCBW (this use case begins when)	The student opens Whoosh Events to look for a certain kind of event
TUCEW (this use case ends when)	The student sees the upcoming events that have the tag they chose
Preconditions	The student has a web browser and an internet connection
Postconditions	The student sees the filtered list, 20 events at a time; nothing in the system has changed
Traces to requirements	FR-01, FR-02, FR-05, FR-07, FR-11, QR-01, QR-02, QR-03, QR-06

Actor input / action	**System response**
	(0) The system shows the first 20 upcoming events, soonest first. Each shows its title, date, start time, location, organizer and tags. It also shows all available tags and the time the list was last updated.
(1) The student chooses a tag (for example “Career”).	(2) The system shows only the upcoming events that have that tag, soonest first, and how many match (for example “14 events”).
(3) The student reaches the end of the events shown.	(4) The system adds the next 20 matching events, or says “No more events” when all are shown.

Alternate flow	**At step**	**Condition**	**What happens**	**Ends or rejoins at**
A1	(1)	The student chooses	The system shows the events that	Rejoins (2)

		more than one tag	have any of the chosen tags	
A2	(2)	No upcoming event has the chosen tag	The system says “No upcoming events match this tag” and offers to clear the filter	Ends
A3	(0)	The last update is more than 2 hours old (a source site could not be reached)	The system shows the events from the last successful update, with “Last updated at” and the time	Rejoins (1)
A4	(2)	The student clears the chosen tags	The system shows all upcoming events again	Rejoins (0)

UC-03 View Event Details

Primary actor	Student
Supporting actors and external systems	The event’s source site (UTD Calendar or Nebula), when the student opens the original page
TUCBW (this use case begins when)	The student sees an event in the list they want to know more about
TUCEW (this use case ends when)	The student sees the event’s full details, or its original page
Preconditions	The student has a web browser and an internet connection

Postconditions	The student has seen the event's details; nothing in the system has changed
Traces to requirements	FR-03, FR-07, FR-09, FR-12, QR-04, QR-05

Actor input / action	**System response**
	(0) Shows the event list. Each event shows its title, date, start time, location, organizer, tags and the name of its source site
(1) Selects an event	(2) Shows the event's full description together with all of the fields above, and a link to the event's original page
(3) Asks to see the original page	(4) Opens the event's page on its source site

Alternate flow	**At step**	**Condition**	**What happens**	**Ends or rejoins at**
A1	(2)	A field (for example the location) is blank on the source site	Shows "Not listed" for that field	Rejoins at (3)
A2	(2)	The event is listed on both source sites, so it was merged into one entry	Names both source sites and gives a link to each	Rejoins at (3)
A3	(2)	The event ended while the student was looking at the list	Shows the details marked "This event has ended"	Ends

UC-02 Share an Event with a Friend

Primary actor	Student
Supporting actors and external systems	The friend who receives the link
TUCBW (this use case begins when)	The student finds an event they want to tell a friend about
TUCEW (this use case ends when)	The friend sees that event's details on Whoosh Events
Preconditions	The student and the friend each have a web browser and an internet connection
Postconditions	The friend sees the current details of the shared event; nothing in the system has changed
Traces to requirements	FR-07, FR-09, FR-10, FR-12, QR-03, QR-04

Actor input / action	**System response**
	(0) The system shows the event list; each event offers "Copy link".
(1) The student asks to copy an event's link.	(2) The system copies a link that opens that event and says "Link copied".
(3) The friend opens the link (the student sends it to them outside our site).	(4) The system shows that event's full details: title, date, time, location, organizer, tags, description, source site and a link to its original page.

Alternate flow	**At step**	**Condition**	**What happens**	**Ends or rejoins at**

A1	(2)	The browser does not allow copying	The system shows the link so the student can copy it by hand	Rejoins (3)
A2	(4)	The event has already ended	The system shows the details marked “This event has ended”, plus upcoming events with the same tags	Ends
A3	(4)	The event was removed from its source site after the link was shared	The system says “This event is no longer listed” and shows upcoming events from the same organizer	Ends
A4	(4)	The event’s time or place changed after the link was shared	The system shows the updated details marked “Updated since this link was shared”	Ends

4.3 Self-review log

Use case	**Check from the Lesson 13 review checklist**	**Defect found**	**How it was fixed**
UC-01	Q2, Q3, Q7 and Q8: step (0), one turn per row, a testable postcondition,	The P1 version had no step (0), the whole flow in one line, an untestable	Rewritten as steps (0) to (4), one turn per row, with a testable

	alternates named by their step	postcondition (they close the website) and an alternate with no step	postcondition; A1 to A4 each name the step they branch from
UC-01	Q1 and Q6: are actors roles that take part, and are preconditions true assumptions?	P1 listed Organizations and Clubs as actors, though they never act, and assumed events exist on the source sites, which can be false	Actors removed; the precondition became alternates A2 (no matching event) and A3 (a source site could not be reached)
UC-02	Q4: is every item the actor supplies or sees an attribute in the domain model?	Alternate A4 needs the time the link was shared, and no class held it	Added the class SharedLink (shared at) to the domain model in 5.1
UC-03	Q4 (same check)	Alternate A2 needs one original page address per source site for a merged event, but Event had room for only one	Added the class SourceListing (original page address), one or two per Event
Refresh Event Data (first draft of 4.1)	Q9: is there background processing that should not be here?	It runs on a timer; no actor starts it, sees it or waits for it	Not expanded as a use case; freshness stays a quality requirement (QR-03) and shows up to the student as alternate A3 of UC-01
All three	Q5 and Q10: does each end with something the actor can see, and does any row name a technology, a store or a screen widget?	None found	No change needed

5 Analysis Models

Chapter 7, second half. What has to exist inside the system for the use cases in section 4 to happen, and how those parts talk to each other.

5.1 Domain model

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
Event	One happening a student can attend, as Whoosh Events lists it (merged into one entry when both source sites list it)	Title, description, date, start time, end time, location, status (upcoming, ended, removed), last changed at	Hosted by 1 Organizer; labeled with 0..* Tag; listed as 1..2 SourceListing; pointed to by 0..* SharedLink
Tag	A topic label a student can filter by, such as Career or Social	name	Labels 0..* Event
Organizer	The club, department or office that runs an event	name	Hosts 0..* Event
SourceListing	The record of an event as one source site publishes it	Original page address, first seen at	Belongs to 1 Event; published by 1 SourceSite
SourceSite	An official UTD site we collect events from (UTD Calendar, Nebula club calendar)	Name, address, last checked at, last successful update at	Publishes 0..* SourceListing
SharedLink	A link a student copied to send one event to a friend	Shared at	Points to 1 Event

5.2 Object interaction models

5.2.1 UC-01 Find Events by Tag

(a) Non-trivial steps.
Non-trivial: step (0), because the system must fetch the upcoming events, the tags in use and the time of the last update; step (2), because it must find the events with the chosen tag, order them and count them; step (4), because it must fetch the next 20 matches or decide there are none. Steps (1) and (3) are the student's own actions; each appears below only as the message that starts the next step.
Objects: EventBrowser is the interaction object that runs the student's visit and is in control throughout. EventCatalog is the interaction object that holds the collected Events and their Tags (5.1). SourceSite is the domain class from 5.1.
(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
0a	EventBrowser	Asks for the first 20 upcoming events, soonest first	Today's date, the page size of 20	EventCatalog
0b	EventBrowser	Asks for the tags in use	none	EventCatalog
0c	EventBrowser	Asks when each site was last updated successfully	none	SourceSite
0d	EventBrowser	Shows the events, the tags and the last-updated time	The event list, the tag list, the last update time	Student
1	Student	Chooses a tag	The tag name	EventBrowser
2a	EventBrowser	Asks for the upcoming events with that tag, soonest first	The tag name	EventCatalog

2b	EventBrowser	Counts the matching events	The matching events	EventBrowser (itself)
2c	EventBrowser	Shows the matching events and how many match	The matching events, the count	Student
3	Student	Reaches the end of the events shown	How many events are shown	EventBrowser
4a	EventBrowser	Asks for the next 20 matching events	The tag name, how many are already shown	EventCatalog
4b	EventBrowser	Adds the next 20 matching events	The next events	Student

(c) Analysis sequence diagram.

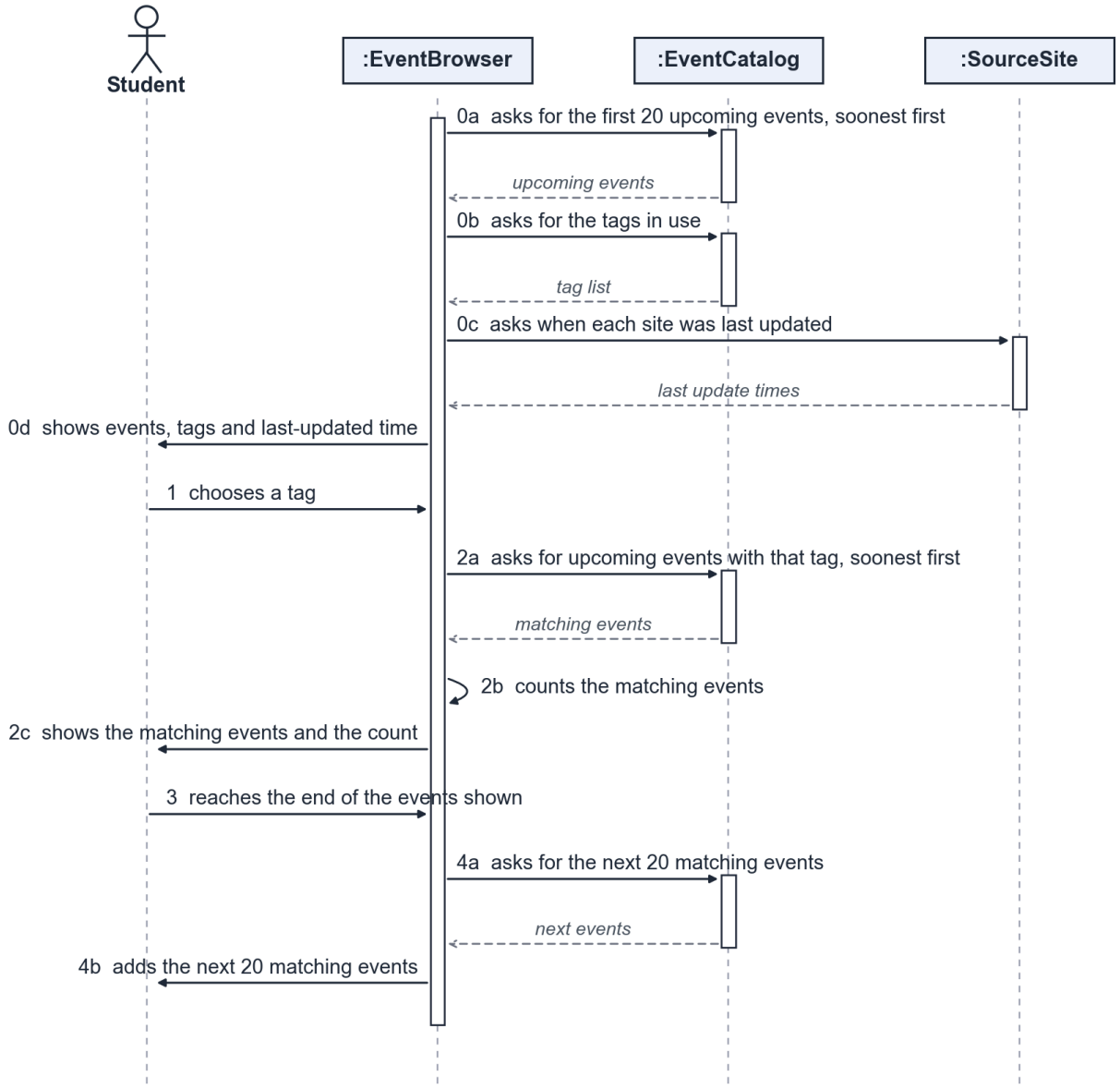


Figure 5.2.1 Analysis sequence diagram for UC-01; message numbers match the scenario table

5.2.2 UC-02 Share an Event with a Friend

(a) Non-trivial steps.

We chose UC-02 because it is our riskiest use case: alternates A2 to A4 depend on how fresh our copy of the source sites is (QR-03, QR-04). Non-trivial: step (2), because the system must make a link that identifies the event and record when it was shared; step (4), because it must find the event the link points to, check that it is still upcoming and whether it changed since it was shared, and only then show it. Trivial: step (0) is the same event list as UC-01 step (0),

already modelled in 5.2.1. Steps (1) and (3) are the student's and the friend's own actions; each appears below only as the message that starts the next step.
Objects: EventBrowser (from 5.2.1) is in control; here it serves the student and, when the link is opened, the friend. LinkRegistry is the interaction object that keeps each SharedLink (5.1). EventCatalog holds the collected Events (5.1).

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	Student	Asks to copy an event's link	The chosen event	EventBrowser
2a	EventBrowser	Asks for a link to the event	The chosen event	LinkRegistry
2b	LinkRegistry	Records the link with the time it was shared	The event, the current time	LinkRegistry (itself)
2c	EventBrowser	Copies the link and says "Link copied"	The link	Student
3	Friend	Opens the link	The link	EventBrowser
4a	EventBrowser	Asks which event the link points to and when it was shared	The link	LinkRegistry
4b	EventBrowser	Asks for the event's current details and status	The event	EventCatalog
4c	EventBrowser	Checks the event is still upcoming and unchanged since it was shared	Status, last changed at, shared at	EventBrowser (itself)
4d	EventBrowser	Shows the full details and the link to the original page	Title, date, time, location, organizer, tags, description, source site, original page address	Friend

(c) Analysis sequence diagram.

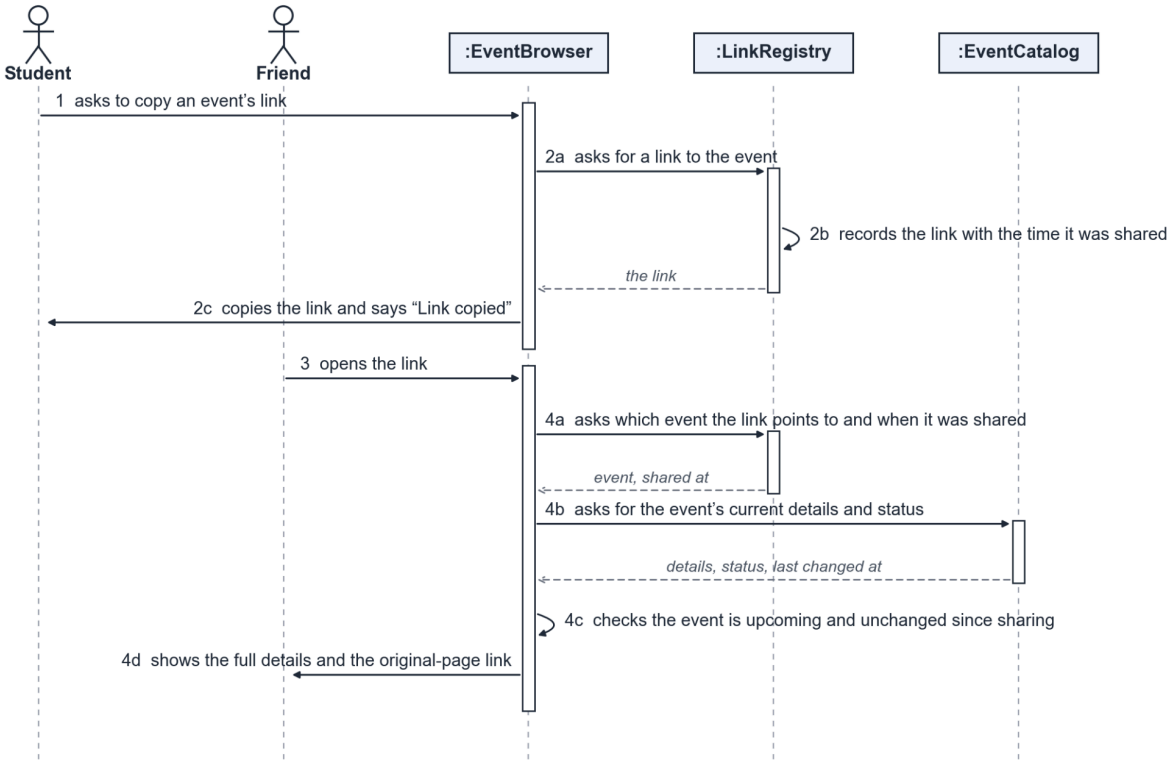


Figure 5.2.2 Analysis sequence diagram for UC-02; message numbers match the scenario table

5.3 Consistency cross-check

Check	Result (pass, or what failed)	Fixed in
K1 Every item in an actor column (4.2) is an attribute in the domain model (5.1)	pass	
K2 Every object on a sequence diagram is a domain class or a declared interaction object	pass	

K3 Every message on a sequence diagram corresponds to exactly one scenario-table row	pass	
K4 Every non-trivial step in 5.2 (a) has at least one scenario-table row	pass	
K5 Every analyzed use case traces to at least one functional requirement	pass	
K6 The last messages of each diagram make the use case postcondition true	Fail (the postcondition for UC-01 says the student sees the filtered list, but figure 5.2.1 displays the last message 4b, which says that the website “adds the next 20 matching events”)	Fixed in 4.2, changed the wording of the postcondition to match the 5.2.1 diagram

6 High-Level Design (Architecture)

Chapter 8. The rule now reverses: a high-level design that does not say where things run, what they are built from and what that choice cost has not been done.

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01	Performance	A student opens the event list, or applies a tag filter, on a 10 Mbps connection with a	C3 website on Vercel	Shows the first 20 event cards, or the filtered list.	Page finishes loading within 3 seconds on at least 19 out of 20 loads. The filtered list appears within 2 s on at least 19 of 20 tries (QR-02).	QR-01,QR-02

		50 ms latency.				
QAS-02	Timeliness	An organizer posts a new event on the UTD Calendar or nebula site	C1 Event collector and the stored events.json	The event is collected, saved and shown on the site.	Within 2 hours for at least 9 of 10 new events	QR-03
QAS-03	Availability	One source site cannot be reached during a scheduled refresh	C1 Event Collector and C2 Event Storage	Events from that site's last good refresh stay in events.json and stay on the site	100% of that site's previously listed events still shown after the failed refresh	QR-06

Constraints shaping the architecture: CON-01 (users cannot create events, so the site is read-only), CON-02 (only the two source sites), CON-03 (robots.txt rules), CON-04 (free hosting only).

6.2 Architectural style and alternatives

Style considered	**Drivers it serves**	**What it costs**	**Verdict**
Repository (shared data file) with a scheduled batch collector; browsers talk to a static site (client-server)	QAS-01, QAS-02, QAS-03, CON-04	Events can be up to one refresh old. The site redeploys on each refresh. The collector must remove duplicates (FR-03).	Chosen

Client-server with live fetch: the site calls both source sites on every page view	QAS-02 Always current	Page speed depends on two outside sites (breaks QAS-01). No stored copy, so a source outage gives an empty list (breaks QAS-03). Needs an always-on server (CON-04).	Rejected
Event-driven / microservices: separate services connected by a message queue	Scaling and easy change	The source sites send no push events, so there is nothing to react to. Several hosted services will not fit a free plan (CON-04). Too much for a 6-person team to build by December.	Rejected

We chose the repository style because the 3 scenarios all depend on keeping a saved copy of events. The collector (C1) writes events.json on a schedule and the website (C3) only reads it, so the page speed in QAS-01 does not depend on the source sites. An hourly refresh (ADR-02) meets QAS-02, which needs new events within 2 hours. A failed refresh leaves the saved file unchanged, which meets QAS-03. Live fetching has no saved copy and cannot meet QAS-01 or QAS-03. Event-driven or microservice designs add parts and cost that CON-04 and our team size don’t allow.

6.3 Component view

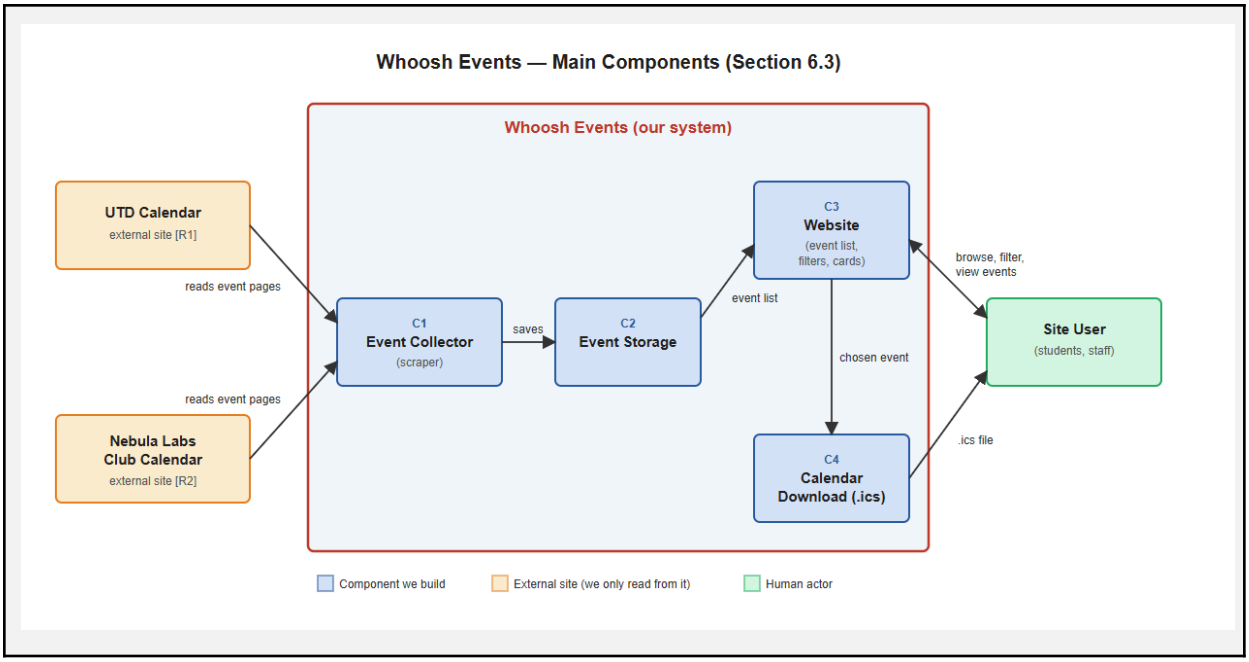


Figure 6.3 Component diagram: components, provided and required interfaces, external systems

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
C1	An event collector that runs on a specific timing and collects event information from other websites.	Duplicated event records	Viewable, public events from Nebula Labs' Club Calendar and UTD Calendar.	UC-01, QR-03, QR-04, FR-03, FR-05, CON-02, CON-03, ADR-03
C2	Keeps records of the events taken from C1.	List of events that can be sorted and filtered.	Event details from C1	UC-01, QR-04, QR-06, ADR-03

C3	Website to display the information collected.	Displays the events	Events from C2, C4 .ics download	UC-01, QR-01, QR-02, QR-05, FR-01, FR-02, FR-04, FR-07 to FR-12, ADR-03
C4	A download that turns a selected event into a .ics file that a user can easily import to their calendar.	.ics file	Event Details from C2	UC-01

6.4 Deployment view

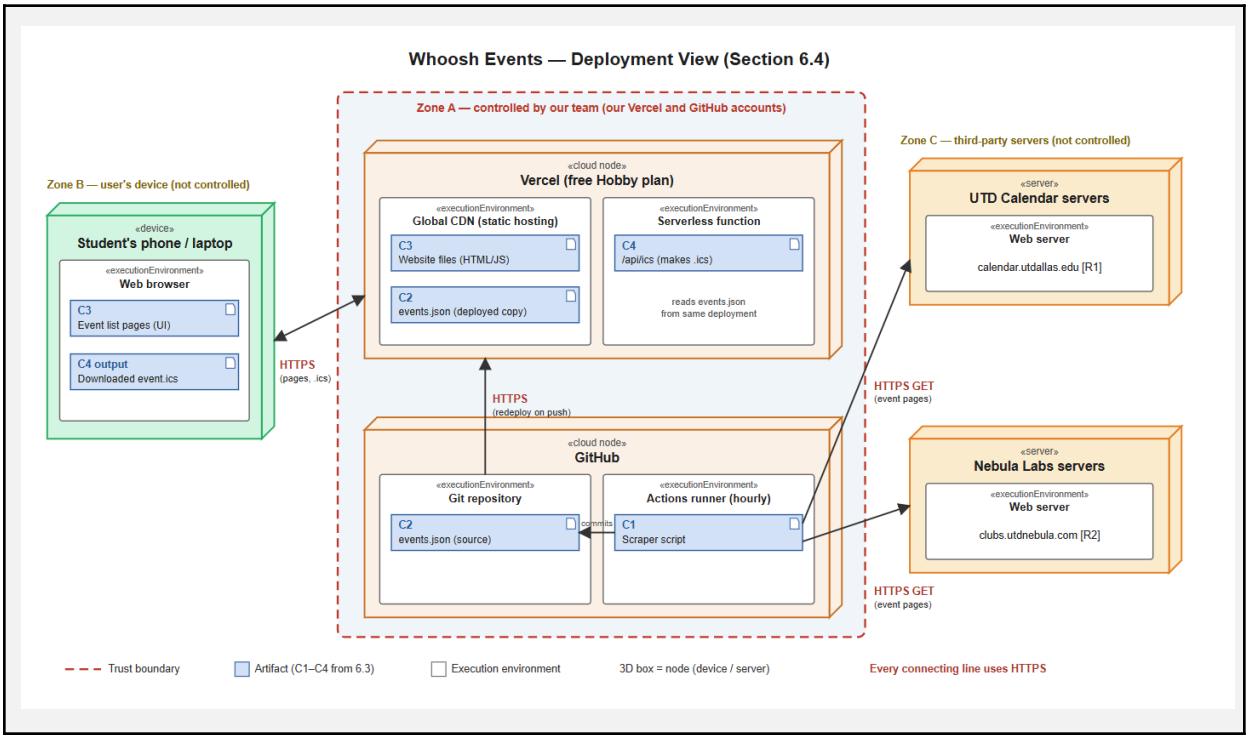


Figure 6.4 UML deployment diagram: nodes, execution environments, deployed artifacts, communication paths with protocols, trust boundaries

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
Student's Pho	A web browser page that lists all events and can receive a downloaded .ics file.	Vercel (HTTPS)	B: user device

ne/laptop			
Vercel	Global CDN that hosts our website files and a copy of events.json.	User's browser (HTTPS), GitHub	A: controlled by us
GitHub	A repository holding events.json and containing the actions runner that can execute our scraper.	UTD Calendar and Nebula Labs	A: controlled by us
UTD Calendar	calendar.utdallas.edu [R1]	Github Action Run	C: third party
Nebular Labs	clubs.utdnebula.com [R2]	GitHub Action Run	C: third party

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	**Component that owns it (6.3)**	**Note (split, merged, external)**
Event	C2 Event Storage	C1 creates and merges events. C3 and C4 only read them.
Tag	C2 Event Storage	Stored with the event. C1 assigns tags and C3 offers them as filters.
Organizer	C2 Event Storage	Stored with the event. C1 reads it from the source page.
SourceListing	C2 Event Storage	Saved with the event. C3 links to it.
SourceSite	C2 Event Storage	Split. The real sites (R1 UTD Calendar, R2 Nebula Labs) are external and we

		only read them. C2 keeps our record of each site: address, last checked time and last successful update time.
SharedLink	C3 Website	The link is created and opened in the website.
EventBrowser (interaction object)	C3 Website	Runs the user's visit and is in control. C3 is the only component the user talks to.
EventCatalog (interaction object)	C2 Event Storage	Merged with the stored Events and Tags. It is read access to C2, not a separate component.
LinkRegistry (interaction object)	C3 Website	Merged with SharedLink.

6.6 Design principles applied

Principle	**Where it shows in your design**	**Evidence (component, interface, ADR)**
Separation of concerns	Collecting events, storing them, showing them and making a calendar file are four separate jobs.	C1 Event Collector, C2 Event Storage, C3 Website, C4 Calendar Download
Information hiding	Only the collector knows how the UTD Calendar and Nebula pages are laid out. The website never reads a source site and only sees stored events.	6.3: only C1 connects to R1 and R2. 6.4: the website reads the deployed events.json only

Low coupling	The collector and the website share one file, events.json, and never call each other. If a source site changes, only C1 changes.	GitHub Actions writes events.json to the repository and Vercel redeploys. ADR-01
High cohesion	Each component does one thing. C4 only turns one chosen event into an .ics file. C3 only displays and filters events.	6.3: C4 takes the chosen event and returns an .ics file
Design for change	A third event source would be a new reader inside C1, with nothing else changing. The refresh interval and the host are recorded decisions, so they can be swapped.	6.3: C1 is the only component tied to a source. ADR-02, ADR-03
Keep it simple (KISS)	No accounts, no database server and no user-created events. One scheduled job, one JSON file and a static website on a free plan.	6.4: Vercel and GitHub Actions only. 1.2: out of scope list

7 Design Rationale

7.1 Architecture decision records

ADR-01 Architectural style

Status	Accepted

Date and owner	11/04/2026- Habib Mashhood
Context (drivers)	QAS-01, QAS-02 and QAS-03 all need a saved copy of the events. CON-02 and CON-03 limit how we may read the sources. CON-04 requires free hosting. CON-01 makes the site read-only.
Options considered	1. Repository style: a scheduled collector writes events.json and the website reads it. 2. Client-server with live fetch from the source sites on every page view. 3. Event-driven or microservices with a message queue.
Decision	Use the repository style . C1 runs on a schedule and writes events.json. C3 reads only that file . C1 and C3 never call each other.
Consequences	Favors QAS-01 (the page reads our own file) and QAS-03 (a failed run leaves the file unchanged). Move: collecting and showing are separate, joined only by events.json. Cost: events can be up to one refresh interval old, and every refresh redeploys the site.
Evidence	Spike S-01 (tests/spikes/S-01/README.md, PR #5) fetched 41 valid events from both sources in 476 ms, and all 41 had the required fields. Open question: the refresh interval is not measured and the GitHub Actions workflow does not exist yet (see ADR-02 and Q1).
Related	ADR-02, ADR-03, QAS-01, QAS-02, QAS-03, R1

ADR-02 Event refresh strategy

Status	Accepted
Date and owner	2026-11-08, Jaiom Rola (Team 05)
Context (drivers)	The app needs current campus events from two approved sources: the UTD Calendar (web page and API) and the Nebula Labs Club Calendar. - Neither source offers a stable, documented feed for our use. We have to scrape HTML, and we have to respect each site's robots.txt. - The app should load fast and keep working when a source is slow, down, or has changed its layout. - The app must show the dataset's last refresh time and source information. - Hosting is static (Vercel), with no always-on backend.

	- QAS-01 (performance), QAS-02 (timeliness: new events within 2 hours, QR-03) and QAS-03 (availability: keep the last good data when a source is down, QR-06).
Options considered	1. Scheduled scrape to static JSON: GitHub Actions runs the scraper every hour and commits events.json to main, and Vercel redeploys. 2. Live scrape per request: a server function scrapes both sources whenever a user loads the page. 3. Manual refresh: a team member runs the scraper by hand and commits the result.
Decision	Use option1
Consequences	- Favors: availability and performance. The site serves static JSON, so page loads don't depend on source uptime or speed. The scraper also puts very little load on the sources. - Move: scraping is separated from serving. Failures are isolated per source, and the keep-last-good fallback covers HTTP errors and HTML layout changes that produce zero events. - Cost accepted: - Data can be up to about six hours stale. - The scraper needs maintenance whenever a source changes its HTML or access rules. - There are extra data commits in git history. - A parser that breaks only partway can still publish partial data. The zero-event fallback doesn't catch that.
Evidence	Spike S-01 (tests/spikes/S-01/README.md) ran with npm run spike:s1: - All three endpoints returned HTTP 200. - The scraper produced 41 valid events against a target of at least 20. - The whole run took 476 ms. - All 41 events had the required fields. - The fallback handled both source errors and HTTP 200 responses that parse to zero events. - The Nebula year-rollover logic stops past dates from being emitted. - Open question: the six-hour interval is a planning choice from the README, not a measured one. Nothing has been measured about how fast event data changes, and the GitHub Actions workflow doesn't exist yet. Repeat the spike if a source changes its structure.
Related	ADR-, QAS-, R- (risk), CH- (change)

ADR-03 Hosting platform

Status	Accepted
Date and owner	11/4/2026 | Austin Ou Yang
Context (drivers)	CON-04 states that we need free hosting for our site. In section 10.4, we also state that we need to be able to maintain separate branches (production and preproduction)
Options considered	1. Vercel - Can allow preview branches, easy deploys through git, and is able to host for free. 2. Netlify - Similar to git, but has less free features and slower support for our framework features. 3. Cloudflare Pages - Has the most free bandwidth, but takes a lot more time to setup.
Decision	Vercel will be used to host our Whoosh Events website.
Consequences	This choice is in favor of QR-01 and CON-04. Move: We connect the Git repo to Vercel, where production(main) connects to the live site and preproduction goes to the preview site. Cost: The Vercel Hobby plan has limits on scheduled jobs and may not allow us to run our scraper hourly.
Evidence	none yet. Open Question: does Vercel Hobby (free plan) allow our scraper to be run at least every 2 hours?
Related	CON-04, QR-01, QR-03, QR-06

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
Fast page load (QR-01) and reliability when a source site is down (QR-06)	Repository style: a scheduled collector writes events.json and the website reads only that file, instead of fetching the sources on every page view.	Events can be up to one refresh interval old, and every refresh redeploys the site.	ADR-1
Freshness (QR-03, new events within 2	The collector runs on a schedule every hour on	Events can be up to one hour old, the	ADR-2

hours) and reliability when a source is down (QR-06)	GitHub Actions, reads both source pages and keeps the last good data for a source that fails	scraper needs maintenance when a source changes its pages, and a parser that breaks halfway can still publish partial data.	
Low cost (CON-04) and fast page load (QR-01)	Host the website on Vercel, connected to the Git repository. The main branch is the live site and the preproduction branch is the preview site.	The free Vercel Hobby plan limits scheduled jobs, so it may not run the scraper every hour or two.	ADR-3

8 Prototype Evidence: Architecture Spike

Spike ID and title	S-01 Read both approved event sources and produce normalized JSON
Risk or decision it addresses	Layout-change risk in 10.3; ADR-02 (event refresh strategy); QAS-02 and QAS-03
Question (answerable yes or no)	Can our scraper read both approved sources and produce valid normalized JSON with at least 20 usable events?
Set-up	Node.js with the built-in fetch and Cheerio. Reads the UTD Calendar web page, the UTD Calendar API (for comparison only) and the Nebula Labs Club Calendar, and writes normalized events to events.json. Folder: /tests/spikes/S-01.
How it was measured	Run npm run spike:s1 from the repository root. The script prints the HTTP status, fetch and parse time, events per source, duplicates removed and the number of valid events (id, title, date, source and sourceUrl all present).

Result (numbers)	Yes. HTTP 200 from all three sources. UTD web page 21 events, UTD API 10, Nebula 20. 41 valid events (target 20), 0 duplicates, 476 ms for the whole run. Missing fields: UTD page 6 times and 7 locations, Nebula 20 locations. All 21 UTD page events were on one day (Nov 6).
What it changed or confirmed	Confirms ADR-02: scheduled scraping of both sites works, and one run takes under a second, far inside the 2-hour limit in QAS-02. Added a fallback to the last good data when a source returns HTTP 200 but zero events (QAS-03), and a year rollover for Nebula dates. Time and location can be missing, so the site must show events without them.
Where to see it	/tests/spikes/S-01 (spike1.js, README.md, events.json); PR #5, commits d2180e1 and 2755b54

9 Traceability

9.1 Extended traceability matrix

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
FR-01	UC-01	Event, SourceListing	C2, C3	ADR-01, ADR-03	S-01
FR-02	UC-01	Event, Tag	C3	ADR-01, ADR-03	None yet (website development in progress)
FR-03	UC-03	Event, SourceListing, SourceSite	C1, C2	ADR-02, ADR-03	None yet (website development in progress)
FR-04	UC-06	SourceSite	C3	ADR-03	None yet (website development in progress)

FR-05	UC-01, UC-05	Event, SourceListing, SourceSite	C1, C2, C3	ADR-02, ADR-03	None yet (website development in progress)
FR-06	UC-04	Event	C3, C4	ADR-01, ADR-03	None yet (website development in progress)
FR-07	UC-01, UC-02, UC-03	Event, Organizer, Tag, SourceListing	C3	ADR-01, ADR-03	None yet (website development in progress)
FR-08	UC-05	Event, Organizer	C3	ADR-01, ADR-03	None yet (website development in progress)
FR-09	UC-02, UC-03	Event, SourceListing	C3	ADR-01, ADR-03	None yet (website development in progress)
FR-10	UC-02	SharedLink, Event	C3	ADR-01, ADR-03	None yet (website development in progress)
FR-11	UC-01	Event	C2, C3	ADR-01, ADR-03	None yet (website development in progress)
FR-12	UC-02, UC-03	Event, SourceSite, SourceListing	C3	ADR-01, ADR-03	None yet (website development in progress)
QR-01	UC-01	Event	C3	QAS-01, ADR-01, ADR-03	None yet (website development in progress)
QR-02	UC-01	Event, Tag	C3	QAS-01, ADR-01, ADR-03	None yet (website development in progress)
QR-03	UC-01, UC-02	Event, SourceSite	C1	QAS-02, ADR-02	None yet (website development in progress)

QR-04	UC-01, UC-02, UC-03	Event, SourceListing	C1, C2	ADR-02	None yet (website development in progress)
QR-05	UC-03	Event	C3	ADR-01	None yet (website development in progress)
QR-06	UC-01	Event, SourceSite, SourceListing	C2	QAS-03, ADR-02, ADR-03	None yet (website development in progress)
CON-01			C3	ADR-01	None yet (website development in progress)
CON-02	UC-01, UC-03	SourceSite, SourceListing	C1	ADR-02	None yet (website development in progress)
CON-03	UC-01	SourceSite	C1	ADR-02	None yet (website development in progress)
CON-04			C1, C2, C3, C4	ADR-03	None yet (website development in progress)

9.2 Gap analysis

Gap	**Items**	**What you did about it**
Requirements with no component	N/A (All requirements have at least 1 component)	

Components with no requirement, QAS or ADR	N/A (All components have a requirement, QAS, or ADR)	
Use cases not analyzed (and why that is acceptable)	UC-03 to UC-06 (no sequence diagram in 5.2)	Acceptable: UC-03 uses the same objects as UC-02 step 2, and UC-04 to UC-06 are single lookups with no alternate flow that changes the design (see 4.1).

10 Project Plan (updated; was P1 section 4)

10.1 Roles and working agreement

Name	P1 Role	P2 Responsibility	Decision rule
Austin Ou Yang	Assist all sections	Repository owner and submitter. Owns 6.3, 6.4, ADR-03, 10.4 and the Vercel and GitHub setup. Submits the final PDF on Canvas.	Unchanged
Jaiom Rola	Sections 4 and 5, help with 1	S-01 spike (section 8), ADR-02, 10.3 risks and 11 open questions.	Unchanged
Ahmed Fouad	Section 3	Sections 4, 5.1 and 5.2 (use cases, domain model, sequence diagrams), 0.2 and the P1 fixes in 1 to 3	Unchanged

Sinan Can Demir	Section 4	6.5, 6.6, 7.2, 10.1, 10.2, 10.5 and 12 (signatures). Reviews Jaiom's spike PR.	Unchanged
Habib Mashhood	Section 2 and parts of 1	6.1 and 6.2 and ADR-01 (architecture decisions)	Unchanged
Atharva Thombare	Section 2	5.3, 9.1 and 9.2 (consistency check, traceability, gap analysis)	Unchanged

10.2 Plan to P3

Milestone	**Date**	**Owner**	**Done when**
UTD Calendar collector	Nov 20	Jaiom	FR-05 is met for UTD Calendar: 10 upcoming events picked from the source all appear in our event list
Nebula collector with last-good-data fallback	Nov 20	Habib	FR-05 is met for Nebula, and FR-03 is met: 3 events listed on both sources each appear exactly once. Old events stay if the site is down.
Scheduled collection job and Vercel deploy	Nov 22	Austin	The job writes events.json and Vercel redeploys on its own
Event list page with pagination	Nov 24	Ahmed	FR-01 is met (20 or more events counted on one page) and FR-11 is met (the next 20 load at the end of the list)
Filters	Nov 27	Atharva	FR-02 is met: for 5 different tags, every event shown has the tag and no upcoming event with it is missing. FR-08 is met for 3 organizers.

Details page, share link and .ics download	Nov 29	Sinan	FR-09 is met (5 event cards show the source description), FR-10 is met (5 copied links open the same event) and FR-06 is met (the .ics file downloads)
Automated tests	Dec 2	Jaiom	Collector and filter tests pass
Mobile and keyboard check	Dec 4	Atharva	Defects found are fixed and rechecked
"How to host an event" page	Dec 4	Ahmed	FR-04 is met: one working link per source site (2) to its official event submission page
Integration check of the whole site	Dec 6	Habib	The site works end to end on preproduction
P3 document checked against P2	Dec 7	Sinan	Every change to P2 is recorded in 0.2
Final review and submission	Dec 8	Austin	P3 is submitted on Canvas

10.3 Risks (updated)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
Event sites change and switch layouts, meaning the parser may not be able to extract info from events anymore.	low	high	Jaiom team - find a backup plan if parser cannot get events
There are issues with some events	low	high	jaiom - have a moderator or create a filter

on different sites being misleading or only partially filled out.			
Several team members working in the same files can cause conflicts to merge edits or cause one person's edits to overwrite another.	medium	medium	jaiom - create pull requests on github and have at least one reviewer

10.4 Repository

Public Repository URL: https://github.com/AO2-Git/whoosh-events
Our folder for p2 is layered like this: /docs/p2, /docs/p2/diagrams, and our spikes our located in /tests/spikes. Our branch and pull request rule is that every person makes their own branch for p2, then creates a pull request to merge into preproduction, then once everything is finalized in preproduction, we issue a pull request to main.

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed
Austin Ou Yang	Claude	Generated the 6.3 component diagram and 6.4 deployment diagram; drafted the 6.3 component table, the 6.4 node table and shared-responsibility paragraph, and the ADR-03 hosting	Checked every component, requirement ID (FR/QR/CON) and source against P1 sections 1.2, 1.4, 2.1 and 3. Cross-checked 6.3 against 6.4. This caught two errors that were then fixed: a missing protocol label on one 6.4 line, and a missing C2 → C4 data flow in 6.3. Hosting claims were compared against the cited web sources.

		decision record; compared Vercel with Netlify and Cloudflare Pages	
Atharva Thombare	Google Notebook (Gemini)	***Section 9.1***: Drafted the extended traceability matrix. ***Section 5.3:*** Generated the list of items/objects for easier readability. Formatted the .md files for my commits	***Section 9.1:*** Checked each entry of the table with the respected section to verify the correctness. Made some changes to the table that was drafted due to it missing some components when cross-referenced. ***Section 5.3:*** Checked each generated item/object that needed to be cross-checked and noted whether each check was a pass/fail. The Ai model was not firm in its response to the pass/fail, so that part was disregarded and manually done.
Sinan Can Demir	Claude Code (Sonnet 5.5)	Drafted sections 6.5, 6.6, 7.2, 10.1, 10.2 and 10.5 and my row in 0.1 (committed as d0935fd, 01ca14e and cd4ccbd in PR #6). Helped me read teammates' PRs (#2, #4 and #5, including the S-01 spike code), check my sections against the template guidance and the working document, and draft my review comments on PR #2 and PR #5, including the approval on PR #5.	I compared every row of 6.5 against the classes in 5.1, the interaction objects in 5.2 and the components in 6.3, and had it rewritten so each object has one owning component, with notes for the split and merged cases. I rebuilt 10.2 with the template's four columns and tied each "Done when" to the acceptance criteria in 3.1. I checked 10.1 against my P1 section 4.1 and the team's work. I checked the 6.6 evidence against 6.3 and 6.4. I read and approved each review comment before posting it from my own account; the PR #5 approval was posted as drafted. I left the 7.2 cells that depend on ADR-01 and ADR-02 blank until those ADRs existed.
Ahmed Fouad	Claude (Anthropic), through Claude Code	Drafted the text of 2.2, 3.4, 4.1 to 4.3, 5.1 and 5.2 and the P1 fixes in 1.1 to 3.5. Wrote the Python script that	I collected the 7 student answers in 2.2 myself. Checked every use case step, class and message against the Lesson 13 and 14 slides, checked that each message number on the sequence

		draws the context, use case, domain model and sequence diagrams (docs/p2/diagrams). Checked our sections against the P2 marking guide and the Lesson 13 and 14 checklists. Commits ee07f0d, 27d6550, 83927f7 and 31fa6c3 carry a Claude co-author line.	diagrams matches its scenario-table step, and checked that every commit and PR cited in 0.1 exists in the repository.
Habib Mashhood	Claude	Drafted 6.1 (QAS-01 to QAS-03), the 6.2 style table and paragraph, ADR-01, and my row of the 7.2 trade-off summary from our P1 requirements and the 6.3 and 6.4 sections.	Checked each QAS number against QR-01, QR-03 and QR-06 in section 3.2. Checked that the style choice matches the components in 6.3 and the nodes in 6.4. Compared ADR-01 with ADR-03 for consistency.
Jaiom Rola	Claude code	Helped with ADR-02 and drafting spike folder and architecture. Helped draft question for section 11	I created tests and went through every function and return to make sure it was correct

11 Open Questions (updated; was P1 section 5)

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q1	Is 1 hour refresh	Jaiom rola	Jaiom - Nov- 16	open/in progress	1 hours

	the right one				
Q 2	Do the robots.txt rules and terms of use for both sources allow scraping at this frequency?	Jaiom rola	Ahmed - Nov 16	open/ in progress	allowed
Q 3	How do we detect a partial parse failure? The fallback only catches errors and zero-event results.	Jaiom rola	Austin - Nov16	Open / in progress	Zero event detection is enough

12 Baseline v1.0 Declaration (was P1 section 6)

We declare this document, version 1.0, the baseline for P2. Sections 1 to 9 describe what we will build in P3. Any later change to them is recorded in section 0.2 with its reason.

Team member	Signature or initials	Date

Sinan Can Demir	SCD	11/08/2026
Austin Ou Yang	AO	11/08/2026
Atharva Thombare	AT	11/08/2026
Ahmed Fouad	AF	11/08/2026
Habib Mashhood	HM	11/08/2026
Jaiom Rola	JM	11/08/2026